

TP2 — Concevoir un pipeline NetOps avec Git, validation et déploiement

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



Figure 1: Pipeline CI/CD à 4 étages

Ce TP construit un vrai pipeline CI/CD à 4 étages (lint → validate → test → deploy) sur le dépôt partagé NexaCorp. L'étape de validation échoue réellement si le contenu est incorrect, et l'étape de déploiement est protégée par une approbation manuelle, pour simuler une promotion contrôlée vers la production.

Prérequis

- TP1 validé : dépôt cloné, branches main / staging / dev créées, environnement production protégé
- Python et pip installés et accessibles dans Git Bash (voir Étape 2 en cas de problème de PATH)

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour1-tp2
```

Étape 2 — Installer yamllint

```
pip install yamllint
yamllint --version
```

Sous Windows, si pip ou yamllint ne sont pas reconnus (command not found), c'est que le dossier Scripts de Python n'est pas dans le PATH de Git Bash. Solution durable, à faire une seule fois :

```
SCRIPTS_DIR=$(python -c "import sysconfig; print(sysconfig.get_path('scripts'))")
WIN_SCRIPTS_DIR=$(cygpath -w "$SCRIPTS_DIR")
powershell.exe -Command "
\$current = [Environment]::GetEnvironmentVariable('Path','User')
if (\$current -notlike '*$WIN_SCRIPTS_DIR*') {
    [Environment]::SetEnvironmentVariable('Path', \$current + ';' + $WIN_SCRIPTS_DIR)
}
"
```

Fermez et rouvrez complètement Git Bash après cette commande, puis retestez `pip --version` et `yamllint --version`.

Étape 3 — Créer un fichier de règles yamllint

```
cat > .yamllint << 'EOF'
extends: default
rules:
  line-length:
    max: 120
  truthy:
    allowed-values: ['true', 'false']
EOF
```

Étape 4 — Tester le lint en local

```
yamllint .
```

Le fichier `inventory/group_vars/all.yml` créé au TP1 doit passer sans erreur bloquante (seul un warning `missing document start` sur `.yamllint` lui-même est normal, il n'est pas bloquant).

Erreurs fréquentes sous Windows : `wrong new line character: expected \n ou no new line character at the end of file` viennent de fins de ligne Windows (CRLF) dans le fichier YAML. Corrigez avec :

```
sed -i 's/\r$//' inventory/group_vars/all.yml
```

Et pour que ça ne se reproduise pas sur tout le dépôt, ajoutez un fichier `.gitattributes` à la racine :

```
cat > .gitattributes << 'EOF'
* text=auto eol=lf
*.yaml text eol=lf
EOF
```

```
*.yaml text eol=lf
EOF
git add .gitattributes
git commit -m "Forcer les fins de ligne LF pour le projet"
```

Étape 5 — Écrire le pipeline CI/CD complet

```
mkdir -p .github/workflows
cat > .github/workflows/netops-pipeline.yml << 'EOF'
name: NetOps Pipeline

on:
  push:
    branches: [dev, staging, main, '*-jour*-tp*']
  pull_request:
    branches: [dev, staging, main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Installer yamllint
        run: pip install yamllint
      - name: Lint des fichiers YAML
        run: yamllint .

  validate:
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - name: Verifier la structure du projet
        run: |
          test -f docs/branching-strategy.md
          test -d roles
          test -d inventory
          echo "Structure du projet valide"

  test:
    runs-on: ubuntu-latest
    needs: validate
    steps:
      - uses: actions/checkout@v4
      - name: Pre-checks / post-checks (placeholder)
        run: echo "Suite de tests reseau - sera completee au Jour 3"

  deploy:
```

```

runs-on: ubuntu-latest
needs: test
if: github.ref == 'refs/heads/main'
environment: production
steps:
  - uses: actions/checkout@v4
  - name: Deploiement (placeholder)
    run: echo "Deploiement ZDD - sera complete Jour 3 / Jour 5"
EOF

```

EOF

Le déclencheur `'*-jour*-tp*'` matche votre convention de nommage de branche (ex. `mouad-mikou-jour1-tp2`).

Étape 6 — Committer, pousser, ouvrir la pull request

```

git add .
git commit -m "Pipeline CI/CD avec lint yamllint, validation, test et deploy proteg
git push -u origin <prenom>-<nom>-jour1-tp2

```

Sur GitHub : ouvrir une pull request `<prenom>-<nom>-jour1-tp2` → dev.

Étape 7 — Vérifier le pipeline dans l'onglet Actions

- lint doit passer au vert (yamllint ne trouve pas d'erreur)
- validate doit passer au vert (les fichiers/dossiers requis existent)
- test doit passer au vert (placeholder)
- deploy doit rester en attente / grisé (car pas sur main : la condition `if: github.ref == 'refs/heads/main'` le fait sauter)

Étape 8 — Provoquer volontairement un échec, puis corriger

Pour vérifier que le pipeline bloque vraiment en cas d'erreur :

```

echo "cle_invalide:valeur sans espace" >> inventory/group_vars/all.yml
git add .
git commit -m "test volontaire erreur yaml"
git push

```

Le job lint doit échouer (rouge). Corriger ensuite :

```

git checkout inventory/group_vars/all.yml
git add .
git commit -m "correction du fichier yaml"
git push

```

Le job lint doit repasser au vert.

Comprendre le pipeline

Chaque job (lint, validate, test, deploy) tourne sur une machine virtuelle Ubuntu neuve, qui récupère le dépôt via `actions/checkout@v4`. La clause `needs: <job`

precedent> fait que chaque étage ne démarre que si le précédent a réussi : d'abord la syntaxe (lint), puis la structure du projet (validate), puis les tests fonctionnels (test), et enfin le déploiement (deploy), qui ne s'exécute que sur main et attend en plus une approbation humaine grâce à l'environnement protégé production. C'est cette chaîne de filtres progressifs qui, dans le scénario NexaCorp, aurait empêché une configuration cassée d'atteindre les utilisateurs sans être testée d'abord.

Validation

- ☐ yamllint . s'exécute en local sans erreur bloquante
- ☐ Le fichier .github/workflows/netops-pipeline.yml contient bien les 4 jobs lint, validate, test, deploy
- ☐ Une pull request <prenom>-<nom>-jour1-tp2 → dev est ouverte
- ☐ Dans l'onglet Actions, lint, validate, test sont verts sur votre branche
- ☐ deploy reste en attente / grisé, car pas sur main
- ☐ Le test volontaire d'erreur YAML fait échouer le job lint (rouge), et la correction le repasse au vert